DEPARTMENT OF INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Master's Thesis in Informatics

RGB Lidar-based Self-Supervised Scene Flow for 3D Object Detection

Özcan Burak Tomruk

DEPARTMENT OF INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Master's Thesis in Informatics

RGB Lidar-based Self-Supervised Scene Flow for 3D Object Detection

RGB Lidar-basierter Selbstüberwachter Szenenfluss für die 3D-Objekterkennung

Author:	Özcan Burak Tomruk
Supervisor:	Prof. Dr.-Ing. Alois Knoll
Advisor:	Emec Ercelik
Submission Date:	16.05.2022

I confirm that this master's thesis in informatics is my own work and I have documented all sources and material used.

Munich, 16.05.2022 Özcan Burak Tomruk

Acknowledgments

To begin with, I would like to express my sincere thanks to Emec Ercelik for advising me throughout this journey and providing essential insights at every stage of my thesis. Additionally, I'd want to say thanks to Prof. Dr.-Ing. Alois Knoll for providing me with the chance to work on this thesis. Last but not least, special thanks to my family for supporting me throughout my education.

Abstract

Nowadays, interest in autonomous driving technologies has risen dramatically as artificial intelligence, deep learning, and other areas improve. The ability to properly perceive the surrounding environment is a need for autonomous vehicles' safety. Cameras, RADAR, and LiDAR are among the self-driving sensors used in autonomous vehicles, but LiDAR has earned a lot of interest due to its long range, high-speed resolution, and ability to produce clear 3D representations of the target. Existing 3D object detection methods, however, rely on massive volumes of annotated LiDAR point cloud data. LiDAR-based systems are generally data-hungry, demanding the labeling of a vast volume of data for training and assessment. Due to the sparsity and irregularity of point clouds, as well as the more complicated interactions involved in this technique, annotating this kind of data is very challenging. Furthermore, relying solely on supervised learning restricts the use of the learned model. In this thesis, we have a built-in self-supervised training system[1] that solves these difficulties, and we want to develop numerous strategies to improve the performance of self-supervised scene flow estimation as a pretext task. These new strategies mainly consist of combining camera and LiDAR-based features in scene flow estimation. We use the shared 3D detection backbone, which predicts 3D scene flow using self-supervised learning and cycle consistency. After that, we use the pretrained shared backbone in our downstream task. NuScenes datasets are used to run experiments, and qualitative results are given.

Kurzfassung

Heutzutage ist das Interesse an autonomen Fahrtechnologien dramatisch gestiegen, da sich künstliche Intelligenz, Deep Learning und andere Bereiche verbessern. Die Fähigkeit, die Umgebung richtig wahrzunehmen, ist eine Voraussetzung für die Sicherheit autonomer Fahrzeuge. Kameras, RADAR und LiDAR gehören zu den selbstfahrenden Sensoren, die in autonomen Fahrzeugen verwendet werden, aber LiDAR hat aufgrund seiner großen Reichweite, Hochgeschwindigkeitsauflösung und Fähigkeit, klare 3D-Darstellungen des Ziels zu erzeugen, großes Interesse geweckt. Bestehende 3D-Objekterkennungsmethoden stützen sich jedoch auf riesige Mengen kommentierter LiDAR-Punktwolkendaten. LiDAR-basierte Systeme sind im Allgemeinen datenhungrig und erfordern die Kennzeichnung einer großen Datenmenge für Schulung und Bewertung. Aufgrund der geringen Dichte und Unregelmäßigkeit von Punktwolken sowie der komplizierteren Interaktionen, die mit dieser Technik verbunden sind, ist das Kommentieren dieser Art von Daten sehr schwierig. Darüber hinaus schränkt das ausschließliche Verlassen auf überwachtes Lernen die Verwendung des erlernten Modells ein. In dieser Arbeit haben wir ein auf selbstüberwachtes Trainingssystem aufgebautes [1], das diese Schwierigkeiten löst, und wir wollen zahlreiche Strategien entwickeln, um die Leistung der selbstüberwachten Szenenflussschätzung als Vorwandaufgabe zu verbessern. Diese neuen Strategien bestehen hauptsächlich aus der Kombination von Kamera- und LiDAR-basierten Funktionen bei der Schätzung des Szenenflusses. Wir verwenden das gemeinsam genutzte 3D-Erkennungs-Backbone, das den 3D-Szenenfluss mithilfe von selbstüberwachtem Lernen und Zykluskonsistenz vorhersagt. Danach verwenden wir das vortrainierte Shared Bacbone in unserer Downstream-Aufgabe. NuScenes-Datensätze werden verwendet, um Experimente durchzuführen, und qualitative Ergebnisse werden angegeben.

Contents

1. Introduction

In today's world, throughout industry and academia, artificial intelligence has been utilized to solve specific problems. People have been paying a lot of attention to the application of artificial intelligence as deep learning and other research have advanced in recent years. Autonomous driving has naturally drawn a lot of interest as one of the most common application scenarios in artificial intelligence. Object detection technology that is efficient and precise is a vital basis for autonomous cars' safety. Autonomous vehicles must be able to consistently distinguish the category and size of objects, as well as information such as their distance, in order to perform environmental perception tasks. As a consequence, 3D object detection research in autonomous driving has gained more attention recently.

Autonomous cars are planned to be equipped with a variety of sensors, including cameras, radar, and lidar sensors, in order to provide a greater understanding of their environment. Although using cameras has benefits such as gathering rich color information and precise measurements of edges and lighting, it also has drawbacks like 3D localization because it performs worse in poor weather circumstances [2]. While 2D RGB images have been widely researched for object detection and tracking, lidar sensors provide more information about the formation, position, and size of objects in a 3D environment with a wider angle than a camera. On the other hand, LiDAR point clouds include less semantic information but provide very accurate 3D localization [3]. Radar sensors have a range of 200–300 meters and use the Doppler effect to determine object velocity. However, the results are much more sparse and less exact in terms of localization than lidar [4]. However with its irregular, unstructured, and unordered features combined with sparsity, the output of lidar sensors, which is in the form of point clouds, provides several obstacles for 3D object detection. Because no one kind of sensor is adequate and the sensor types are complimentary, multimodal datasets like nuScenes[5] which we use in our experiments, are especially important. There are many massive amounts of annotated data that may be utilized in supervised learning algorithms for self-driving vehicles, and it will be explained in detail in the next chapters.

1.1. Motivation

Perception of the environment requires accurate 3D object detection in a point cloud in order to safely drive autonomous vehicles. 3D object detection predicts the 3D bounding boxes of objects in a scene, which is an important step in scene understanding. In order to perform the task, we required huge annotated datasets such as nuScenes[5] or KITTI[6]. They include

a considerable amount of annotated data that may be used to improve the accuracy of 3D object detection in autonomous vehicles trained using supervised learning. Despite the large amount of data with labels, there are certain flaws. To begin with, all of the labels in these datasets are manually annotated, which takes a lot of time and resources. To address the scarcity of labeled data and the limitations of supervised learning, self-supervised learning can be used in tasks like scene flow estimation [7], which does not require a large amount of labeled data. Furthermore, the model may learn the structure of the dataset through self-supervised learning, allowing the trained model to generalize more effectively.

Feature representations generated from LiDAR point clouds are used in LiDAR-based 3D object detection and scene flow networks. Self-supervised learning representations for scene flow may be utilized as an initial representation for 3D object detection. This thesis is focusing on a self-supervised LiDAR scene flow approach to pre-train the backbone of the 3D object detection model so that detecting object results can improve. The model could learn the structure of the point cloud and then reduce the data label needs by pretext tasks.

Apart from lidar, a camera may give important information about the scene since there is beneficial and related sensor fusion research such as CamLiFlow[8] and DeepLiDARFlow[9]. The present scene flow models depend exclusively on LiDAR point clouds. However, the contextual information of the points from the RGB data might help in relating one point set to another in the following frame. As a result, we propose that image features from ResNet[10] and RGB information corresponding to point inputs be used for scene flow, and thus its contribution to 3D object detection be evaluated.

1.2. Outline

This section will explain the structure of the thesis. In the first chapter, this thesis's motivation and main objective are introduced. Then in the second chapter, related works, researches and background of the thesis such as 3D object detection, ResNet[10], self-supervised learning, optical flow and scene flow will be explained. In Chapter 3, our approach will be introduced in detail. Furthermore, experiments with this approach are shown in Chapter 4. In Chapter 5, we evaluate and discuss the results of these experiments. Finally, we made a conclusion and also discussed the future work of our method in Chapter 6.

2. Background and Related Work

First of all, a research study of 3D object detection on point clouds will be provided. After that, one of the most well-known and widely used backbone networks, ResNet, is explained. Furthermore, a literature review on self-supervised learning, optical flow, and scene flow will be presented in order to clarify the methods used in this thesis.

2.1. LiDAR 3D Object Detection on Point Clouds

With the rapid growth in the world of technology, it has become more vital for the automotive sector to keep up with the quick pace, just like any other business. Object detection is a critical component of autonomous driving. Typically, autonomous vehicles are outfitted with a variety of sensors, including cameras, RADAR, and LiDAR. Although Convolutional Neural Networks are the state-of-the-art technology for detecting 2D objects, they perform poorly on 3D point clouds owing to the limited sensor data. As a result, some operation in the middle of the detection in the 3D point cloud is required.

While CNN object detection neural networks presume their inputs are ordered data structures expressed in grid form, point clouds have a permutation invariance trait that prohibits us from simply adopting them. Point clouds are unordered collections of points, suggesting that altering the order of the points has no effect on the three-dimensional objects represented by the point clouds. If neural networks are to directly consume point clouds, this characteristic demands that they are permutation invariant with regard to their inputs. Changes in the sequence of the input lidar points should have no effect on the predicted 3D boxes produced by neural networks.

PointNet[11] is a permutation-invariant neural network optimized for classifying point clouds and semantic segmentation tasks. PointNet[11] is incapable of doing three-dimensional object detection because its classification network assumes that all lidar points in an input point cloud belong to a single object.

Frustum PointNets[12] augments PointNet[11] with a three-dimensional region proposal network in order to partition the three-dimensional input scene into subspaces containing only lidar points of a single object. As a result of this partitioning, a network that is similar to PointNet may be utilized to perform 3D object detection tasks while focusing on the subspace corresponding to a particular 3D proposal region. In Frustum PointNets[12], the proposal net-

work is constructed using three-dimensional projections of two-dimensional bounding boxes predicted by a two-dimensional image-based object detection network. As a result, Frustum PointNets[12] is a multi-modal network which uses image and lidar. In addition, Frustum-Pointpillars[13] is a multi-stage approach for 3D Object Detection using RGB camera and lidar.

The key feature of this type of 3D object detection network is to represent point clouds comparable to images as structured grids so that they may benefit from the current image-based CNN object detection networks. By encoding point clouds as ordered grids, these ordered grid representations may immediately be fed to CNN object detection networks such as Faster-RCNN[14]. A method for gridding point clouds is 3D voxelization, which is accomplished by quantization of the 3D cuboid subspaces surrounding lidar sensors. To implement 3D voxelization, a 3D cuboid encircling a specific lidar sensor is first determined based on the lidar sensor's range and the goal range of a given application. Three-dimensional voxelization converts raw point clouds to a structured grid representation. They may be utilized directly as inputs to CNN object detection networks. This enables approaches for 3D voxelization representation to take advantage of recent improvements in CNN object detection networks.

VoxelNet[15] partitions three-dimensional space into voxels and converts them to a matrix representation that records the point interaction inside a voxel. Convolutional Neural Networks extract complicated information and produce the bounding box's confidence and regression values. SECOND[16] reduces the complexity of the VoxelNet and accelerates sparse 3D convolutions. Moreover, there is also another method for gridding point clouds, which is 2D voxelization. CNN object detection networks using two-dimensional voxelization grids are more computationally efficient than networks processing three-dimensional voxelization grids. Each two-dimensional voxel cell is referred to as a pillar, which is a voxel cell that has the same height as the three-dimensional cuboid voxel representation. Pillar encoders are mappings that take the lidar points contained within a certain pillar and create a feature vector of a fixed size that corresponds to the pillar. PointPillars[3] is a widely used model for pillar encoders.

There are also methods that can combine voxel and point based methods. PV-RCNN[17] integrates both 3D voxel convolutional neural networks and set abstraction based on PointNet[11] to learn more discriminative point cloud features. It makes use of the 3D voxel CNN's fast learning and high-quality proposals, as well as the PointNet-based networks' variable receptive fields.

Centerpoint[18] is capable of detecting and tracking three-dimensional objects in the form of points. It constructs a representation of the input point cloud using VoxelNet[15] or PointPillars[3] backbone network. Then it flattens this representation into an overhead map view and locates the object's center using a typical image-based keypoint detector. It optimizes and enhances earlier anchor-based three-dimensional detectors. HPVR[19] uses an approach that

combines point-wise and voxel-wise features. To lower the computational cost, they enhanced the point-based features with a memory module. They then combine the memory features that are semantically related to each voxel-based feature to create a hybrid 3D representation in the form of a pseudo image, enabling them to effectively locate 3D objects in a single step.

Self-Supervised Pretraining of 3D Features on any Point-Cloud[20] has a similar approach to ours. They describe a simple self-supervised pretaining approach that may deal with any 3D data, like single or multiview, inside or outdoor, obtained by various sensors. They demonstrate that pretraining typical point cloud and voxel-based model designs boosts performance much more and test their models on nine benchmarks for object detection, semantic segmentation, and object classification.

2.2. Residual Networks (ResNet)

Due to vanishing and exploding gradients, training deep neural networks is challenging. To address this issue, skip connections are used. These connections allow activation from one layer to be instantly sent to a layer considerably deeper in the neural network. By doing so, you might actually establish a network that enables you to train extremely deeply in that service, up to a hundred layers deep in certain cases.

By integrating residual blocks, you can train even deeper neural networks, and by stacking many residual blocks together to make a deep network, you may create a resident. Typically, if you choose a network with too many layers, the training error increases, but with ResNets[10], the training error performance can continue to improve as the number of layers increases. Additionally, when retraining a network with more than a hundred or even thousands of layers, you can get the same result, so it is commonly used in many studies nowadays. Residual blocks and intermediate activations enable it to go much deeper than your network, which solves the vanishing and exploding gradient problems and enables you to train much deeper neural networks with no appreciable performance loss. At some point, this will plateau, as it will flatten out and no longer benefit the deeper networks. As it is shown in the [10], ResNet-152 has the lowest training error of the other ResNet versions even though it has more network layers than others.

2.3. Self-supervised Learning

Although acquiring large amounts of data has become easier these days, the labeling of training data still requires manual work, which poses a major barrier to supervised learning, which is heavily dependent on the labeling data. However, self-supervised learning could help to overcome this challenge. In self-supervised learning, the model is fed by unsupervised data, which is then able to learn its characteristic structure and characteristics from the

unlabeled data.

In self-supervised learning, we use a task that we call the "pretext task" to prepare for the learning process. Afterward, we use downstream tasks to fine-tune the interaction. The idea behind pretext tasks in computer vision is that they are designed to ensure that a network that is trained to solve them will learn visual characteristics that can easily be applied to other downstream tasks.

The approach proposed by STRL[21] is based on the spatiotemporal context and structure of the point cloud. It takes as input two temporally correlated frames from a 3D point cloud sequence, transforms them via spatial data augmentation, and self-supervisedly learns the invariant representation. Additionally, the spatio-temporal contextual cues incorporated in 3D point clouds boost the learnt representations considerably.

Self-supervised monocular scene flow estimation[22] is a new monocular scene flow approach with competitive accuracy and real-time performance. They construct a single convolutional neural network that correctly calculates depth and 3D motion from a traditional optical flow cost volume using an inverse problem approach. To make use of unlabeled data, they use self-supervised learning using 3D loss functions and occlusion reasoning. Adversarial Self-Supervised Scene Flow Estimation[23] presents a self-supervised technique in which a network learns a latent measure to differentiate between flow estimate points and the target point cloud. Their adversarial metric learning includes a multi-scale triplet loss on two-point cloud sequences, as well as a cycle consistency loss.

SLIM[24] is a self-supervised motion segmentation signal based on the difference between a robust rigid ego-motion estimate and a scene flow estimate. The system subsequently uses the expected motion segmentation to pay attention to stationary points for the aggregation of motion information in static areas of the scene.

2.4. Optical Flow

Object detection and semantic segmentation, techniques that detect or classify objects at a pixel level for real-time processing of video input, have enabled computers to perceive their surroundings, but since most of these techniques focus only on how objects relate within the same frame, time information is ignored. In a sequence of frames, optical flow describes the motion of objects between successive frames, caused by the relative motion between the object and the camera.

Sparse optical flow offers the flow vectors of particular key characteristics, such as a few pixels indicating an object's edges or corners within the frame, while dense optical flow gives the flow vectors of the whole frame up to one flow vector per pixel. Dense optical flow provides more accuracy, but it is slower and more costly to calculate.

A novel deep network architecture for optical flow, Recurrent All-Pairs Field Transforms[25] which captures per-pixel features, creates multi-scale 4D correlation volumes for all pairs of pixels, and repeatedly updates a flow field using a recurrent unit that conducts correlation volume lookups.

PWC-Net[26] warps the CNN features in the second image using the current optical flow estimate. It then constructs a cost volume from the warped features and features of the initial image, which is processed by a CNN to estimate the optical flow. SelFlow is a[27] self-supervised learning approach for optical flow. Upgrading Optical Flow to 3D Scene Flow through Optical Expansion [28] is an approach for upgrading 2D optical flow to 3D scene flow using dense optical expansion between two views that can be learned from annotated optical flow maps or unlabeled video sequences.

2.5. Scene Flow

Scene flow, as opposed to optical flow, is the three-dimensional motion field of a point in three-dimensional space. The positional change of a point between two successive frames is described by scene flow, which also includes information regarding spatio-temporal information.

[7] proposes a technique for training scene flow that includes two self-supervised losses based on nearest neighbors and cycle consistency. Additionally, [29] investigates an RGB-D-based scene flow model. FlowNet3D[30] uses PointNet++[31] to work directly on points and offers a flow embedding that is calculated in one layer to capture the correlation between two point clouds and then conveyed via finer layers to estimate the scene flow. Flownet3D[30] consists of 3 different modules, which are point feature learning, point mixture, and flow refinement. The feature learning module extracts point features from the pair of point clouds. A flow embedding is derived from the point-wise features via a set of convolution layers in the point mixing module. After that, the flow refinement module creates an approximated scene flow.

Moreover, in Deep rigid instance scene flow[32], the model is based on the idea that the motion of the scene may be constructed by estimating the 3D motion of each vector. They suggested a unique deep-structured model that takes visual cues from optical flow, stereo, and instance segmentation. In PV-RAFT[33], they present a method for estimating scene flow from point clouds that uses point-voxel correlation fields, which capture both local and long-range relationships of point pairs. RAFT-3D[34] deals with the problem of scene flow by estimating pixelwise 3D motion from a pair of stereo or RGB-D video frames. Rigid-motion embeddings, which describe a soft grouping of pixels into rigid objects, are a major feature of RAFT-3D. Therefore, they try to benefit from lidar and depth as input for scene flow estimation.

PointPWC-Net[35], which is based on PWC-Net[26], presents a learnable point-based cost volume without the need for 4D dense tensors. These approaches use a coarse-to-fine strategy, in which scene flow is calculated at a low resolution initially and then upsampled to a high resolution.

There are several studies that focus on combining Lidar and RGB image features to estimate scene flow and optical flow and get noticable results. Therefore, this has an influence on integrating similar approaches to our method in [1]. CamLiFlow[8] is mainly composed of many bidirectional connections between 2D and 3D branches in specified layers. They utilize a point-based 3D branch to better extract geometric features and create a symmetric learnable operator to merge dense image features and sparse point features, which differs from earlier work. Furthermore, DeepLiDARFlow[9] is a revolutionary deep learning architecture that predicts dense scene flow by fusing high-level RGB and LiDAR features at several scales in a monocular configuration.

3. Method

In this study, we have attempted to utilize and develop our technique in [1] which consists of mainly two parts such as self-supervised scene flow estimation and 3D object detection. Our key objective is to enhance the pretext task, which is scene flow estimation, so that it may improve our detection task. Both parts use the same backbone so that our downstream task benefits from the pretext task. In our pretext task, which is self supervised scene flow estimation, we want to train the common backbone with self-supervision because of the limitations and drawbacks of the labeled dataset. We chose to use FlowNet3D[30] with integration of cycle loss in our pretext task in order to extract meaningful and motion-aware point features on the backbone. Furthermore, we used our pre-trained backbone in our downstream task, which is 3D object detection, so that it can detect and discriminate objects more accurately with supervised learning and a smaller annotated dataset. The next parts are focused on a detailed introduction of the methodologies.

3.1. Problem Statement

We intend to train the backbone using self-supervised learning based scene flow estimation to get an improvement in detecting objects. Despite that this is the primary objective, we focus on improving our pretext task, scene flow estimation, by integrating more meaningful features or information than just lidar point clouds, since notable studies like CamLiFlow[8] and DeepLiDARFlow[9] have fused lidar and camera sensors to improve data association in scene flow estimation. Therefore, in our first method, which is introduced in detail in 3.2, we integrated only two sequential image features into the flow head because the input should be two consecutive scenes as it is mentioned in [1]. These features are extracted from pretrained ResNet[10] which is not related to the training dataset. Moreover, in our second methodology, as we discussed in detail in 3.3, we also added RGB information from a specific scene into each point cloud via a custom-implemented projection method. During self-supervised scene flow training, we want to benefit from the point motion representations learned by the 3D feature extractor. In 3D object detection, a tuple is created which includes the class id, bounding box shape, and coordinates of the object in the frame which is mapped from the point cloud. As a consequence, the 3D detection head may make greater use of the spatio-temporal motion representations supplied by the feature extractor backbone in order to get enhanced object detection results.

3.2. Scene Flow Estimation with self-supervision (Pretext)

In our pretext task, we try to train the 3D detector's backbone in order to learn the point cloud structure with unlabeled data using self-supervised learning. We chose to use scene flow estimation as a pretext task and integrated the 3D detector's backbone with the scene flow head in FlowNet3D[30] which is explained in Chapter 2.5. The backbone picks up point representations that indicate object movement patterns by backpropagation of scene flow gradients. While we were training the combined model, we utilized self-supervised cycle consistency loss [7]. As it is mentioned in [1], our main purpose is extracting local features sampled points from consecutive frames using the backbone in the scene flow estimation so that learnt spatio-temporal features may also be utilised to differentiate objects more accurately in the downstream task.

Moreover, there are various skip connections between distinct modules. In our example, we integrated the PointPillars backbone instead of PointNet++[31] as the point feature learning module and also removed the final skip connection into the set upconv layers from the first set upocnv layer.

Self-supervised cycle consistency loss is used to train the combined model. We compute the flow between two time frames in both forward and backward directions, which implies the scene flow will be calculated twice and both ways. The loss of a cycle helps to reduce the mismatch between the first frame and the estimated frame by allowing self-supervised training of the backbone. This means that the 3D detector's backbone might learn the point cloud representation between two successive frames by backpropagating the scene flow gradients during training.

Our shared backbone for both tasks is in PointPillars[3] which accepts a point cloud as an input. The PointPillars network is made up of three components: a Pillar Feature Net, a Backbone, and a Detection Head. The point cloud is turned into many pillars which are partitioned on only the x-y plane after voxelization. Firstly, each point in the cloud which contains a 4 dimensional vector is changed to 9 dimensions in this module to provide information about associated pillars. In Pillar Feature Net, which contains a learnable encoder component that utilizes PointNet[11], the point cloud representation is learned. In addition, the point cloud is converted to a pseudo-image which is the input of the 2D CNN backbone. Then, to extract more useful features, a set of linear layers and a 2D CNN are used. These features are sent into the detection head, which produces the prediction results. The backbone module in our case contains the linear layers of the Pillar Feature Net as well as the 2D CNN layers. Thus, we use this backbone in our self-supervised learning method to estimate the scene flow and the input to the flow head is points with their features.

3.2.1. Integrating Image Features from ResNet to Scene Flow Head

We have tried to input random image features besides sampled features to FlowNet3D[30] in order to investigate the effects of the training result. First of all, random consecutive scene images were selected, which are not in our training data but have similar features. As it is mentioned in Chapter 2.2, adding more layers to the neural network decreases the training error in ResNets. Therefore, we chose the best version of pretrained ResNet, which is ResNet-152, while we were trying to extract features from images.

To use pretrained ResNet-152, we preprocessed the two sequence images, transforming, normalizing, and cropping them, before giving them as input. Then we got the extracted features of both images from the last layer of ResNet-152. In the voxel encoder, voxelized features are sampled randomly and a specific number of points are selected, which is 2048 points. In addition, since we sample features in a voxel encoder, we need to adjust the dimensions of extracted features so that we can combine them.

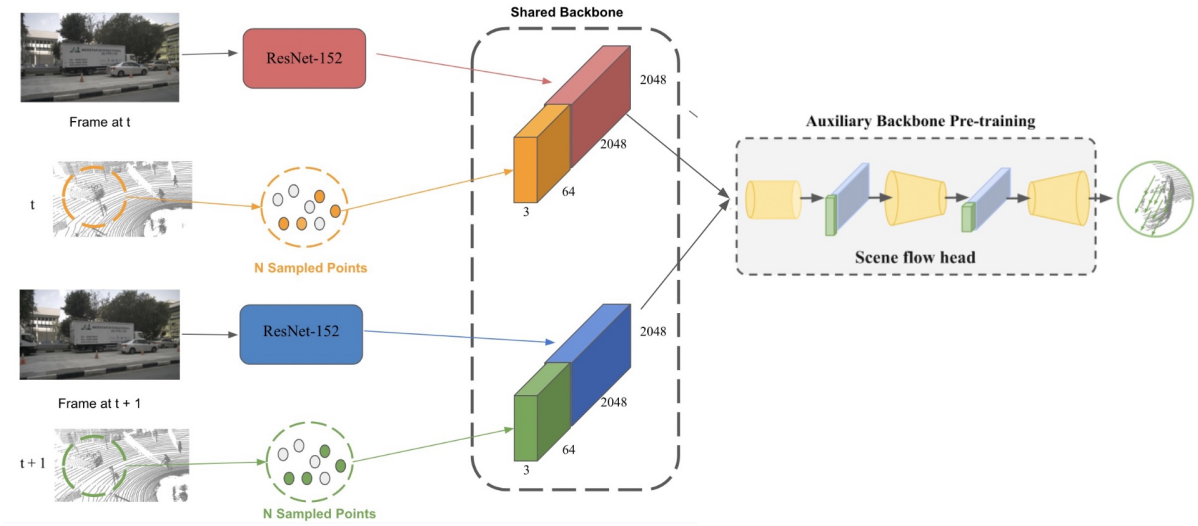

Figure 3.1.: Overview of the integrating image features from ResNet[10] to the Scene Flow Head. The image features are combined with N sampled features from the backbone's point cloud. Shared backbone and flow head is utilized from [1]

As it is shown in Figure 3.1, sampled features have 64 channels and our extracted features have 2048. Finally, we gave these combined features of both scenes to the flow head and updated the flow head configuration. We changed the first convolutional layer and the last

upconvolutional layer of input channel number, which is now 2112 channels according to the new combined features, because there is a skip connection between those layers.

3.2.2. Integrating RGB Point Clouds to the Backbone

This method mainly consists of two parts, which are lidar point cloud to image projection and adding RGB information to the point cloud. We implement a method that uses nuScenes[5] dataset for the projection of point clouds onto images. Furthermore, the config file of nuscenes in the mmdetection3d is changed to load images of each scene into the framework. RGB point clouds were given to the voxelize method as input instead of normal point clouds in the voxel encoder. Therefore, input channels of the voxel encoder in the config file of point pillars are updated according to new point clouds since the RGB point cloud has 7 dimensions instead of 4 as you can see in Figure 3.2.

Lidar Point clouds to Image Projection

Since we have loaded images from each scene corresponding to each lidar point cloud into the mmdetection3d framework, we can construct accurate projections. In nuscenes[5] we have 6 camera angles for each scenes from camera sensor which means we made projections for each angle and combine those at the end so that most of the points in the point cloud have RGB values.

For each projection, we have five steps. Firstly, we transform the point cloud to the ego vehicle frame for the sweep's timestamp by using calibrated sensor information of rotation and translation from the point sensor. The next phase is to transform from the ego to the global frame. Then, for scene image from camera sensors, we transform from global to ego vehicle frame. The fourth phase is to transform from ego to camera. We take the projected image by matrix multiplication and renormalization using camera-matrix as it is shown in Equation 3.1. Finally, we attempted to remove any points that were outside or behind the camera and utilized the masked RGB data that will be discussed in depth in the next chapter.

$$\mathbf{N} = \mathbf{P} \cdot \mathbf{C} \tag{3.1}$$

where:

P = points of lidar point cloud
C = camera intrinsic
N = projected new points

As you can see from the visualization of the projection method of the single camera angle in Figure 3.2, points are projected on the 2D image plane. Then it is converted to a 3D point cloud by combining point cloud and RGB features from projection.

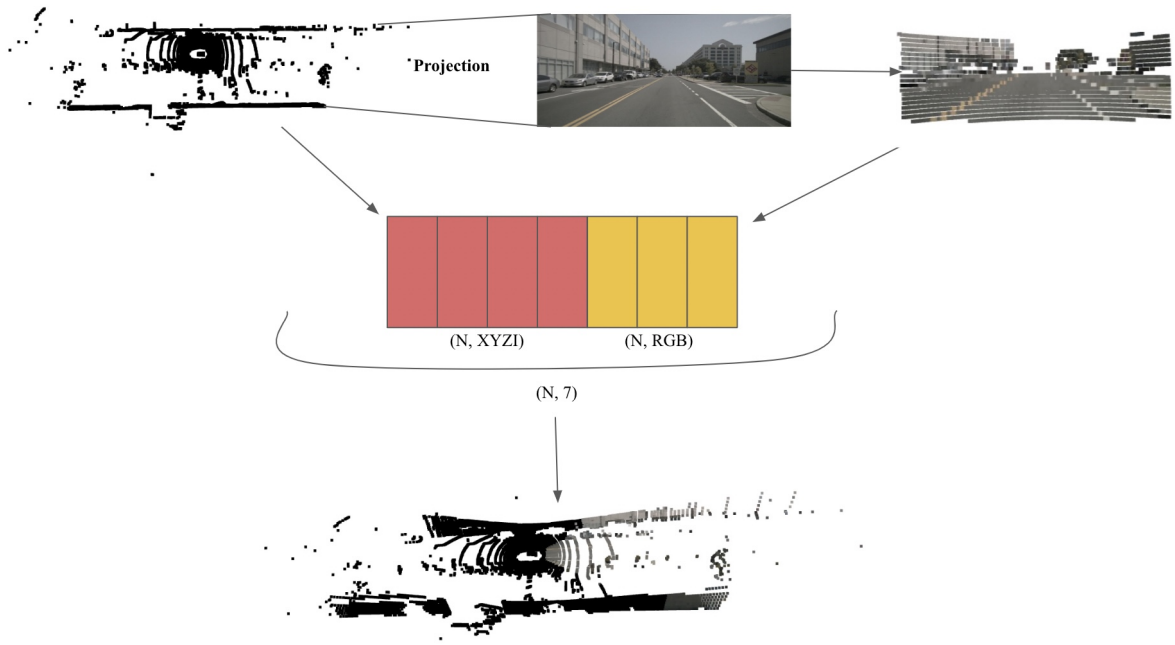

Figure 3.2.: Overview of the point cloud to image projection on single camera angle and converting it to RGB point cloud. Projected points are firstly on 2D image plane then it is concatenated with the original point cloud.

Converting Point clouds to RGB Point clouds

Projected points onto a 2D image plane with their color information were visualized using OpenCV[36] and compared to the original image of the scene to confirm and validate the projection technique was correct. Because a 3D point cloud includes more points than a 2D image, various points in the cloud may be projected to the same point on the image or may not be projected at all during the projection process. Thus, we masked the points that are within the image before integrating RGB information.

$$\mathbf{RGB} = \sum_{i=1}^{6} P_i \tag{3.2}$$

where:

P = RGB point cloud of single camera angle
RGB = Complete RGB point cloud

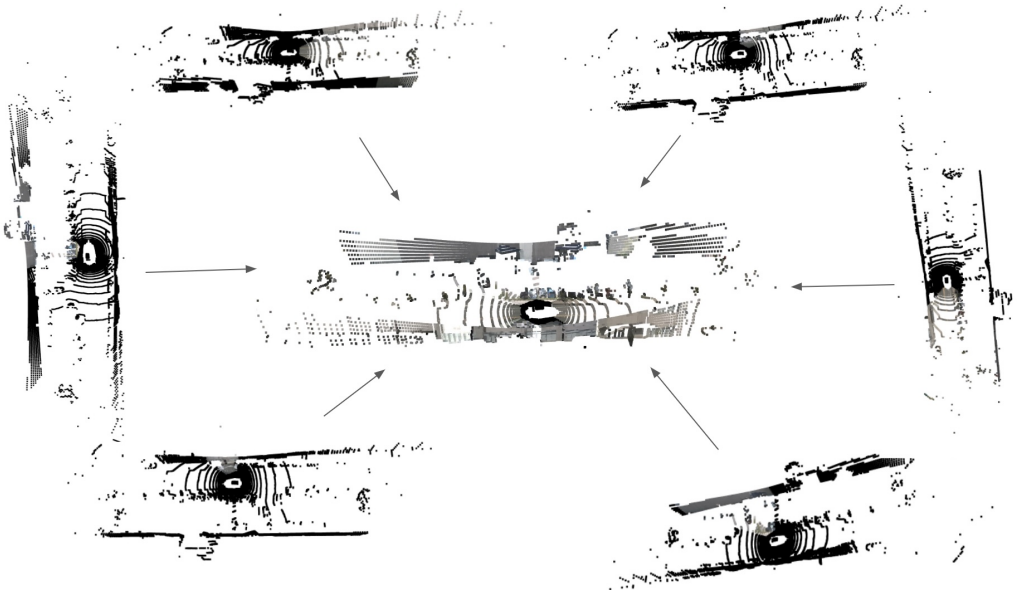

Figure 3.3.: Generating a complete RGB point cloud from all six camera angles of the converted RGB clouds. Front, front left, front right, back, back left, and back right are the camera angles in the nuScenes[5] dataset.

After that, RGB information is integrated to build the coloured point cloud for each of the scene's six camera perspectives, as shown in Figure 3.3. Normally, LiDAR point clouds contain four dimensions: three-dimensional coordinates and intensity values. However, with conversion, each RGB point cloud has seven dimensions, including RGB information.

Once again, OpenCV[36] is used to visualize the 3D lidar RGB point cloud and compare it to the scene to ensure that the RGB cloud was accurate in Figure 3.4 and Figure 3.5. We can confirm that RGB coloring differentiation in the point clouds are accurate. The blue car's location in the point cloud is moved to the right in time frame t to t+1, indicating that the driver moves to the left.

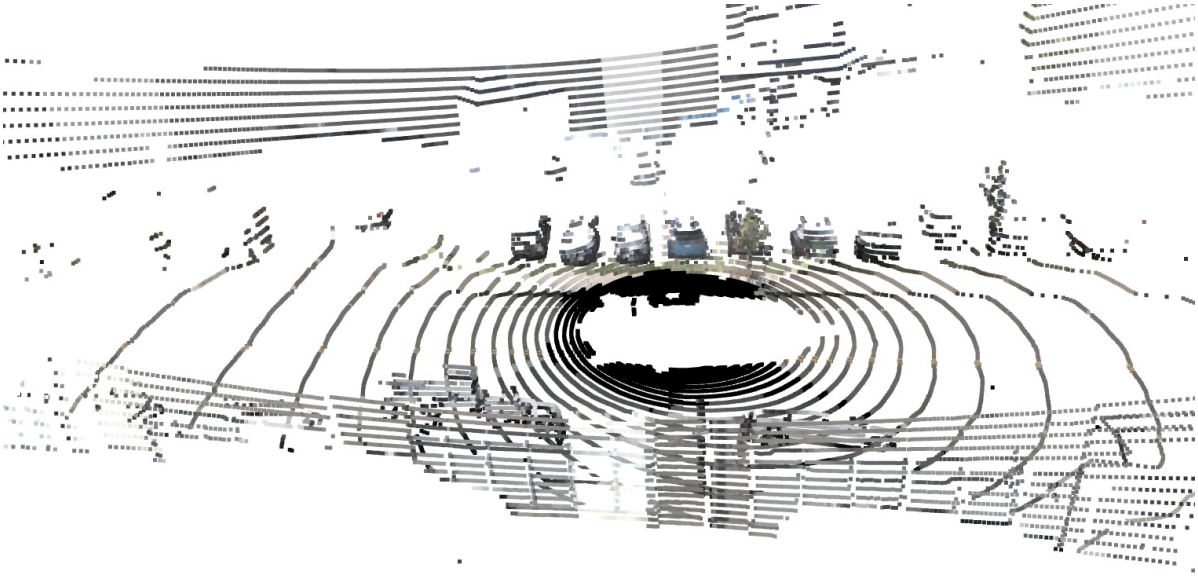

Figure 3.4.: Zoomed in version of complete RGB point cloud conversion of the scene at t. Most of the points have distinct RGB information in the point cloud

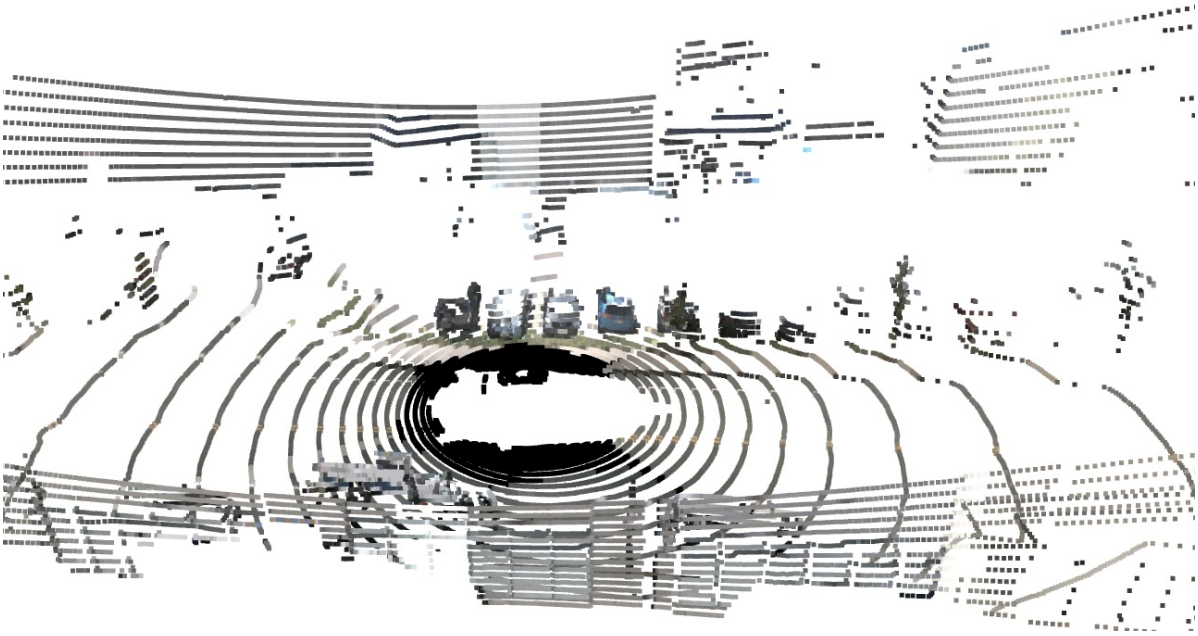

Figure 3.5.: Zoomed in version of complete RGB point cloud conversion of the scene at t+1

3.2.3. Combination of 3.2.1 and 3.2.2

Because those approaches are implemented progressively, we'd want to examine the implications of using them individually and simultaneously on scene flow estimates and detection results. While we combine 3.2.1 with 3.2.2, we need to refactor common parts of the voxel encoder that have been changed before. As it is demonstrated in Figure 3.6, converted RGB point clouds are sent into the voxelize technique in the common detector's backbone and image features are added to the sampled features before being given as input into the flow head.

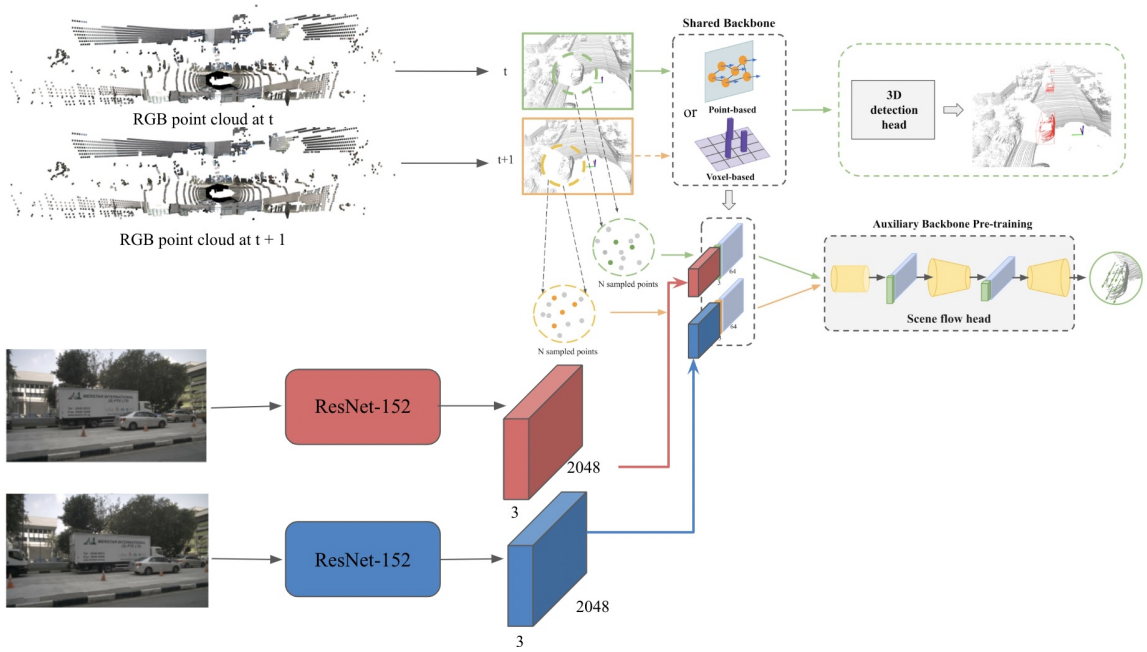

Figure 3.6.: Overview of the whole architecture with our combined methods of integrating image features from ResNet[10] and RGB point clouds. RGB point clouds are directly given to shared backbone. Extracted image features from ResNet are concatanated with N sampled features inside the backbone. Detection and scene flow heads are used from [1]

3.3. 3D Object Detection (Downstream)

As a downstream task, we're interested in 3D object detection as well. Scene flow and detection heads share the same backbone. The scene flow head is only utilized for scene flow

training as pretext task and not for 3D detection training.

PointPillars[3] detection head uses single shot detector(SSD) to perform the 3D object detection. Using a multibox for bonding box regression approach, object localization and classification are completed in a single forward pass of the network. The primary goal is to construct 2D bounding boxes on the features that come from the PointPillars' backbone. Moreover, SSD regresses the bounding box locations using priors and filters out noisy predictions.

We use the pre-trained weights from the self-supervised scene flow training to initialize the backbone weights of the 3D detector. Based on distinct object motion and geometry-aware features, the 3D detection head captures distinguishing spatio-temporal point cloud features. As a result, even after supervised training with a smaller labelled dataset, the 3D detection network can recognize objects more accurately.

3.4. Loss

We use the self-supervised loss described in [7] to train the 3D detector backbone and the scene flow head in self-supervised scene flow loss. For 3D object detection training, we use the same loss functions that we use for 3D detectors, which is PointPillars[3].

3.4.1. Scene Flow Loss

Scene flow loss is actually a combination of two types of losses, which are the Nearest Neighbour (NN) and cycle consistency losses [7] as it is shown in Equation 3.5. The NN loss estimates the Euclidean distance between the converted point and its closest neighbor. As we explained above, we estimate the scene flow for forward and backward in both directions and use it in the cycle consistency loss computation. The total of the two losses constitutes the final self-supervised loss.

$$L_{cycle} = \sum_{i}^{N} \|x_i^{''} - x_i\|^2 \tag{3.3}$$

$$L_{NN} = \frac{1}{N} \sum_{i}^{N} \min_{y_j \in Y} \|x_i^{'} - y_j\|^2 \tag{3.4}$$

$$L_{flow} = L_{cycle} + L_{NN} \tag{3.5}$$

where:

L_{cycle} = Cycle Loss
L_{NN} = Nearest Neighbour Loss
L_{flow} = Scene Flow Loss

3.4.2. 3D detector loss

In our downstream task, we used the default PointPillars'[3] loss for the object detection. The loss from the SECOND[16] research is used in the framework. There were three types of losses. There was a total localization loss, a focal loss for classification, and a total training loss, and these three losses were merged to form the object classification loss and minimize the actual network loss. Therefore, we used the Adam optimizer to optimize the loss function. The orientation is predicted using the softmax classification loss, which is applied to a collection of discrete bins. The focal loss is used as a classification loss for the object classes. For the total loss, all of the loss values are aggregated with their corresponding coefficients, and we utilize the default values from the mmdetection3d[37] repository.

3.4.3. Validation Loss

To examine the flow estimations in the self-supervised scene flow training, we use validation loss. It is mainly based on the calculation of cycle loss as in Equation 3.3, which is the same in the training but this time it based on the dataset used for the validation. We have updated corresponding files in the mmdetection3d framework so that we can test our estimated flow checkpoints on different data than the training dataset. Because we want to test flow estimations, we integrate the shuffle configuration for giving sequential frames to the flow network as we did for integrating flow in [1] and cancel the prediction of bounding boxes, which is not required. To be more explicit, since we utilize sequential input, we calculate each pair's cycle loss and divide it by the size of the validation dataset, as indicated in Equation 3.6.

$$L_{val} = \frac{1}{N} \sum_{i}^{N/2} L_{flow}(i) \tag{3.6}$$

where:

L_{flow} = Scene Flow Loss Equation in 3.5
N = Validation dataset size
L_{val} = Validation Loss

4. Experiments

We analyze our various versions of the self-supervised backbone pre-training technique with PointPillars[3], which is based on voxelization, in the experiments chapter. The improved FlowNet3D as well as the cycle consistency loss are integrated into the PointPillars training pipelines for the self-supervised scene flow task. Firstly, we train the common backbone with a self-supervised scene flow estimation task with the integration of image features and RGB information into the input. Then we utilize the pretrained backbone's weights to initialize in the 3D object detection task. On the nuScenes[5] dataset, we examine our method. Moreover, we have used different portions of the partial training dataset and validation dataset in order to see the effectiveness of our strategy.

4.1. Dataset

Both the self-supervised scene flow estimates and the 3D detection tasks are performed using the nuScenes[5] dataset. Nuscenes[5] dataset is an autonomous driving dataset that contains 700 training and 150 validation drives and includes lidar, radar, and video modalities acquired from a 360-degree viewpoint. A car outfitted with 6 cameras for image captioning, for which we are using the data from these sensors in our projection method, 5 radars, and a lidar sensor gathered the dataset. It consists of 1000 scenes from Boston and Singapore in various weather conditions. Each scene lasts 20 seconds and depicts a range of scenarios that might occur in traffic. Keyframes are annotated frames in the dataset that are recorded at a rate of 2Hz. We mainly use the data from cameras and lidar sensors in our methods.

4.2. Evaluation Metrics

For nuScenes[5] dataset, we mainly focus on the metrics for detection results, which are the average precision (AP) per class, the mean average precision (mAP) across all classes, and the nuScenes detection score (NDS). NuScenes[5] utilize the Average Precision (AP) metric, but instead of intersection over union, they define a match by thresholding the 2D center distance d on the ground plane (IOU), and mAP is the mean of all AP scores. Furthermore, the nuScenes detection score (NDS) is suggested as an evaluation metric which combines the many error kinds into a single scalar score.

4.3. Implementation Details

4.3.1. Repository

The repository we have used is mmdetection3d[37]. MMdetection3d is a PyTorch-based open source object detection toolkit for outdoor and indoor 3D detection. It has a modular design because they decompose the detection framework into individual components, making it simple to put together a customized object detection system by mixing various modules. It supports common 3D object detection datasets and we used the nuScenes[5]. Although many detectors have distinct model structures, they all contain similar components that may be grouped into the following parts: encoder, neck, head, roi extractor, and loss.

In the encoder, which is a voxel based approach in our method, even though the encoder module is PillarFeatureNet, we utilize the HardVFE for nuScenes dataset. To extract feature maps, the backbone is commonly an FCN network. The neck is the component between the backbones and heads. It refines or reconfigures the raw feature maps generated by the backbone, and the Feature Pyramid Network (FPN) is utilized on nuScenes in the neck. In the head module, we add the scene flow head to mmdetection3d and eliminate the 3D detection head since the initial stage in our strategy is self-supervised scene flow learning under unlabeled data. On the nuScenes dataset, we cancel the scene flow head and use Anchor3DHead for 3D object detection. We don't use the RoI extractor, which is responsible for extracting RoI features from feature maps, in PointPillars. Then, we integrate scene flow loss from [7] into mmdetection3d and use it for self-supervised backbone and scene flow learning. To train the downstream 3D detection task, we use the default loss value in mmdetection3d.

4.3.2. Training

To train and evaluate our self-supervised learning for the 3D object detectors technique, we use one GTX 3090 (24GB). Our self-supervised backbone pre-training approach can be used with different 3D detector architectures. We evaluate our method mainly with PointPillars[3]. For the self-supervised scene flow task, we add the modified FlowNet3D[30] as well as the cycle-consistency loss to the 3D detectors' training pipelines, and we integrate the image features from ResNet[10] to the input of the flow head and also RGB point clouds by the implemented projection method to the input of the shared backbone.

Scene Flow head training

The FlowNet3D[30] uses a collection of points from two consecutive frames as input to construct flow vectors. The network's point features are extracted using two cascaded PointNet[11] Set Abstraction modules, each with a three-layer MLP. In the FlowNet3D scene flow head, flow embedding, set conv layers, and set upconv layers are followed by fully-connected

layers for predicting point flow vectors. The point features from the backbone of the 3D detector are sent to the second PointNet Set Abstraction module, which replaces the first PointNet Set Abstraction module. Because we integrate the backbone, only the final set upconv layer from the original PointNet Set Abstraction module that permits skip connections is removed.

The flow head module specifies the FlowNet3D model's parameters for self-supervised scene flow training on the whole nuScenes dataset, while the other components are set to the default value of mmdetection3d. We often use the pre-trained FlowNet3D weights on the supervised FlyingThings3D[38] simulation data to generate our scene flow head weights. The backbone for the PointPillars-based scene flow training is the Adam optimizer with a 0.001 learning rate and voxel encoders. For N = 2048 sampled points, our batch size is 2. On a single Nvidia RTX 3090 GPU, we trained the network for 12 epochs.

3D detection head training

To initialize the weights of the detector's backbone for 3D detection head training, we utilize the weights of the pre-trained backbone based on self-supervised learning. The detection head is initialized using the mmdetection3d default option. The 3D object detection head is trained using training data from the nuScenes dataset. The initial learning rate is e^{-3} with a decay weight of 0.001 and decays at the 20th and 23rd epochs. There are 24 training epochs in all. AdamW is used as the optimizer. We used one Nvidia RTX 3090 GPU for the training.

4.4. Experiment Details

We conducted many tests to demonstrate how our updated self-supervised learning strategy improves 3D object detection performance. Firstly, we have experiments on PointPillars baseline, PointPillars with Flow. Then we run experiments of our methods which are integration of ResNet features, integration of RGB point cloud and integration of ResNet features and RGB point cloud with partial datasets of nuScenes. We compared our techniques to the PointPillars[3] baseline and PointPillars with Flow in [1] on the nuScenes dataset using the following assessment metrics: mAP, NDS, and AP for each category.

We assess the model's performance in all classes, but we focus mainly on these three categories: cars, pedestrians, and barriers. Even though we primarily concentrate on trials using 10% datasets, we also look at the effects of training on 5% and 2.5% partial datasets on the findings, as we did in our ablation research.

In addition, the cycle losses on the validation datesets are used to analyze flow estimation. However, we also attempt to visualize the predicted flows to determine whether our technique works. Our self-supervised scene flow training may produce an accurate estimation of the flow of point clouds between two successive frames, as shown in Figure 4.1.

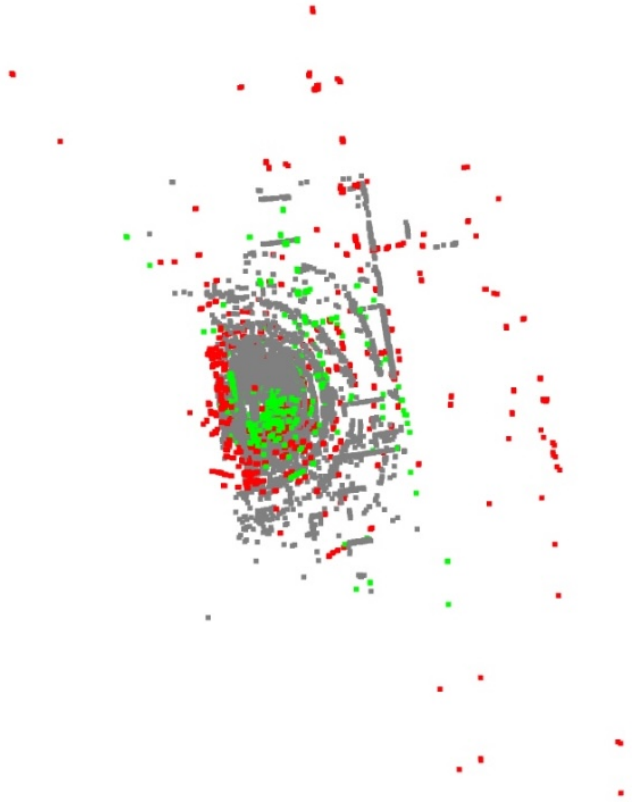

Figure 4.1.: Scene flow estimation using sampled nuScenes point clouds. Red points are from frame t . Frame t + 1's actual points are shown by grey points. From frame t until frame t + 1, our self-supervised scene flow task evaluated the location of the green dots.

5. Results and Evaluation

To assess each technique, we first look at its performance on the pretext task, which is scene flow estimation. As a result, we examine the scene flow task's validation losses separately. Then, based on it, we perform detection experiments. Tables 5.4 and 5.5 in Chapter 5.4 provide all detection outcomes from all approaches. Therefore, in next chapters, we refer to these tables to evaluate each approach and make the most accurate comparisons possible.

5.1. Evaluation of Integrated Image Features from ResNet

In order to perform the downstream task, which is object detection, first we examine the validation losses of all the saved checkpoints in the flow estimation task. Therefore, validation loss is based on the cycle loss we use in our training. This means we determine which checkpoint to use for the detection task. We compare our Scene Flow + ResNet method with only Scene Flow so that we can look at the improvements in the estimated flows.

Epocs	Scene Flow	**Scene Flow + ResNet**
Epoch 1	52.882	52.661
Epoch 2	139.748	116.608
Epoch 3	92.804	113.406
Epoch 4	100.595	53.463
Epoch 5	51.594	89.235
Epoch 6	52.785	**43.069**
Epoch 7	49.419	199.657
Epoch 8	71.142	940.760
Epoch 9	60.279	751.022
Epoch 10	78.059	599.032
Epoch 11	**42.517**	1430.533
Epoch 12	62.018	1119.983

Table 5.1.: Validation loss comparison of Scene Flow and Scene Flow with ResNet

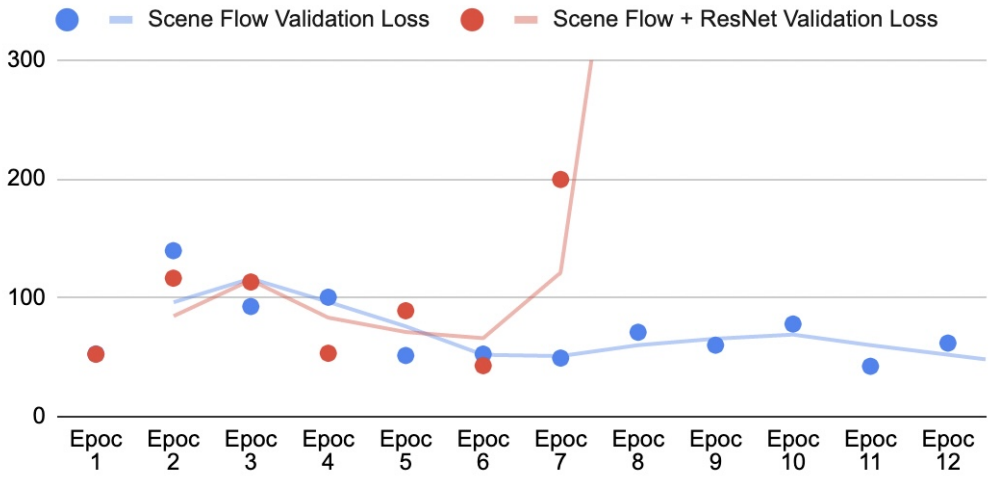

Figure 5.1.: Validation loss comparison of Scene Flow and Scene Flow + ResNet

As it is shown in Figure 5.1, our method's validation loss values are similar or even less until the 6th epoch and it just gets worse after that. Hence, this means that we overfit after the 6th epoch. Therefore, we choose the epoch that has the lowest validation loss value, which is also 6 epoch, as you can see in Table 5.1 and Figure 5.1. Furthermore, we run the downstream task from normal scene flow with the 11th epoch, which has the lowest value at the validation loss.

After pretext task completion and examination, we proceed to the PointPillars[3] based 3D object detection as our downstream task. Weights are initialized from the scene flow estimation task with the best possible checkpoint for both methods. In detection results, the most important metrics are mAP and NDS in nuScenes[5], therefore we compare those values in all methods, as you can see in Table 5.4. Furthermore, we also look at the classes' average precision (AP) values in order to better differentiate methods' performance. We use the 10% nuScenes dataset for the object detection task, and the baseline method achieves better in these two metrics than this technique.

Because we want to use less annotated data as possible in object detection, we chose to train on a 10% dataset of labeled data in order to acquire the best results with benefits of time efficiency. However, we were unable to improve detection performance over the baseline method since the dataset was not large enough to properly train for improved object differentiation. On the other hand, we accomplish at least as well as the PointPillars + Flow in our ResNet version.

5.2. Evaluation of Integration of RGB point clouds method

Epocs	**Scene Flow with RGB**
Epoch 1	**369.431**
Epoch 2	880.491
Epoch 3	925.698
Epoch 4	1117.365
Epoch 5	1034.352
Epoch 6	1443.09
Epoch 7	**658.95**
Epoch 8	1067.684
Epoch 9	1234.156
Epoch 10	1049.46
Epoch 11	948.253
Epoch 12	1719.133

Table 5.2.: Validation loss table of Scene Flow with RGB

We also want to train the method of integration of RGB point clouds into shared common backbones separately and examine its effects on the detection results. We have the same procedure as in other training; therefore, we calculate the validation losses and choose to use the lowest loss value.

Although it seems that the initial epoch has the lowest value, we believe this may have occurred by chance since points are randomly sampled in scene flow estimation, so we use both of them for the detection task and compare with the best result. The 7th epoch in the table 5.3 has a superior performance than the first epoch in the downstream task. Hence, we use those results in the Table 5.4 and our assumption is acceptable.

As it is shown in Table 5.4, detection result of the integrated RGB point cloud approach achieves better from baseline method in NDS metric and slightly poorer in mAP metric, which might also be a significant improvement. We can't be certain since we only used 10% of the dataset, but we can claim that our integrated RGB point clouds technique may perform better than Flow and Flow + ResNet independently.

5.3. Evaluation of 3.2.1 and 3.2.2 combined

Epocs	**Scene Flow with RGB and ResNet**
Epoch 1	**498.196**
Epoch 2	672.817
Epoch 3	719.173
Epoch 4	780.736
Epoch 5	1183.173
Epoch 6	1147.127
Epoch 7	**806.57741**
Epoch 8	1412.383
Epoch 9	1109.289
Epoch 10	1750.537
Epoch 11	1452 241
Epoch 12	1923. 482

Table 5.3.: Validation loss table of Scene Flow with RGB and ResNet

We want to investigate the efficacy of a combination of these two approaches once they've been included into scene flow estimation and trained independently. While we compute the validation losses of all epochs after scene flow estimation, once again we get a similar pattern as in the integrated RGB point clouds version. This means that for our downstream task, we choose the epoch with the second lowest validation loss value, which is Epoch 7.

On the other hand, after we changed the PointPillars' configuration, such as omitting multiple sweeps, the cycle loss and validation loss have increased. The reason for higher losses might be loading less point cloud data at once since we need to increase the accuracy of the our projection method. As a consequence, approaches that integrate RGB point clouds have higher validation losses than others, which is shown in Table 5.3.

Even though flow validation losses are higher, using the combined approach of our two methodologies, we are able to achieve slightly improved results in terms of both key metrics compared to other approaches, as shown in Table 5.4. Furthermore, our new approach has comparatively better precision (AP) results in most of the classes in Table 5.5. As a consequence, we may conclude that combining these strategies has a better impact on detection outcomes than utilizing them separately.

5.4. Detection Results of all methods

In order to evaluate our new methods, first of all we obtain detection results of PointPiilars baseline and PointPillars with Flow on 10% nuScenes dataset and then we conduct experiments on each of our new approaches as you can see in Table 5.4 and 5.5. As a consequence, we can compare and assess our approaches against these outcomes.

Method	Flow	ResNet	RGB	mAP	NDS
PointPillars Baseline	✗	✗	✗	0.1956	0.295
PointPillars with Flow	✓	✗	✗	0.172	0.280
PointPillars with Flow and ResNet	✓	✓	✗	0.183	0.285
PointPillars with Flow and RGB	✓	✗	✓	0.1923	**0.2979**
PointPillars with Flow and RGB and ResNet	✓	✓	✓	**0.1993**	**0.3095**

Table 5.4.: Detection results of all methods with 10% nuScenes dataset

Method	Car	Truck	Bus	Trailer	Cons. V.	Pedestrian	Motor-cycle	Bicycle	Traffic Cone	Barrier
PointPillars Baseline	**0.683**	0.143	0.056	0.018	0	0.579	0.094	0.001	0.111	0.271
PointPillars with Flow	0.643	0.124	0.056	0.018	0	0.447	0.078	0	0.097	0.256
PointPillars with Flow and ResNet	0.664	0.137	0.066	0.022	0	0.449	0.09	0.001	**0.122**	**0.275**
PointPillars with Flow and RGB	0.678	0.142	**0.074**	**0.029**	0	0.57	0.069	**0.002**	0.114	0.245
PointPillars with Flow and RGB and ResNet	0.678	**0.15**	**0.074**	0.018	0	**0.583**	**0.113**	0.001	0.116	0.259

Table 5.5.: Class detection results of all methods with 10% nuScenes dataset

5.5. Ablation Study

Results and Evalutation of different nuScenes parital datasets

Due to the high expense of manual annotating the autonomous driving data, 3D annotated datasets are restricted in size for most real-world applications. In this ablation, we explore the efficacy of our self-supervised pre-training strategy in the absence of annotated data. Besides training with 10% of the nuScenes dataset, all methods that we examine are trained using 5% and 2.5% of annotated data randomly selected from the nuScenes training set, respectively. As a reminder, the whole nuScenes training set is still being used to train our self-supervised scene flow.

Method	**mAP**	**NDS**	Car	Truck	Bus	Tr.	C.V.	Pedes-trian	Motor-cycle	Bi-cy-cle	Traffic Cone	Barrier
PointPillars baseline	0.1338	0.2364	0.574	0.084	0.024	0	0	**0.364**	**0.032**	0	0.065	0.196
PointPillars with Flow	0.1341	0.2363	0.583	0.084	0.034	0	0	0.359	0.019	0	0.07	0.199
PointPillars with Flow and ResNet	0.**1347**	**0.241**	0.583	0.084	0.034	0	0	0.359	0.019	0	0.07	0.199
PointPillars with Flow and RGB	0.1318	0.2353	0.579	0.074	**0.037**	0	0	0.356	0.026	0	**0.074**	0.171
PointPillars with Flow and RGB and ResNet	0.1332	0.2392	**0.586**	0.084	0.021	0	0	0.349	0.025	0	0.06	**0.205**

Table 5.6.: Detection results with 5% nuScenes dataset

Method	**mAP**	**NDS**	Car	Truck	Bus	Tr.	C.V.	Pedes-trian	Motor-cycle	Bi-cy-cle	Traffic Cone	Barrier
PointPillars baseline	0.0859	0.1846	0.478	0.026	0.001	0	0	0.266	0	0	0.032	0.056
PointPillars with Flow	**0.0898**	**0.1907**	**0.492**	**0.037**	**0.004**	0	0	0.263	0	0	0.032	**0.069**
PointPillars with Flow and ResNet	0.0897	0.1855	0.491	0.035	0.003	0	0	**0.275**	0	0	**0.037**	0.057
PointPillars with Flow and RGB	0.0884	0.1816	0.479	0.028	0.001	0	0	0.274	0	0	0.035	0.067
PointPillars with Flow and RGB and ResNet	0.088	0.1827	0.489	0.032	0.001	0	0	0.266	0	0	0.029	0.064

Table 5.7.: Detection results with 2.5% nuScenes dataset

The detection scores and average precision values for all classes of the methods are once again examined in detail. However, Table 5.6 and 5.7 show that there are no significant variations in detection outcomes for these partial datasets for any of the approaches. Hence deriving any conclusions about these smaller datasets is difficult. When training with a larger dataset, we can observe the benefits of our novel ideas more clearly since the network is better able to distinguish between objects. A larger dataset along with our combined technique of image features and RGB point clouds may have achieved higher accuracy in 3D object detection.

6. Conclusion and Future Work

6.1. Conclusion

For the scene flow estimation, this thesis proposes fusion of the camera and lidar based on a self-supervised backbone training strategy in order to improve pretext task and our main approach in [1]. We utilize self-supervised learning to train the backbone of the 3D detector using unannotated data.

For our pretext task, we picked scene flow estimation as our self-supervision method, using cycle consistency. We have integrated image features from ResNet[10] to scene flow head and corresponding RGB information to the point clouds of the scene in order to enhance scene flow estimation and hence the influence of shared backbone in the detection task. We extracted image features from two unrelated scenes using pretrained ResNet. We also add RGB information to each associated point by mapping the point cloud to a relevant image of the scene. These new and beneficial features, we believe, may have an impact on the pretext task. We also look at the effectiveness of each approach independently and in combination in order to achieve the best results.

The backbone of the 3D detector may learn the structure of the point cloud via self-supervised learning, making it simpler for the chosen 3D detector to predict the objects. PointPillars' 3D object detection performance can be improved with our approach, as shown in numerous experiments on the nuScenes dataset. We ran the tests using different nuScenes partial datasets to assess how successful each variant of labeled data is in the downstream task. In conclusion, despite the lack of data, we try to demonstrate that our modified self-supervised pre-training approach may enhance the 3D object detection task.

6.2. Future Work

In this thesis, our method has demonstrated promising outcomes, but it is not a completely proven approach. Training using all datasets may offer a clearer understanding of the efficacy of our methodology since we have outcomes mostly on partial datasets due to the time limitation. Our technique might be improved in various ways, such as the integration of ResNet features. We might add features from all the scenes in the dataset sequentially since we have only included image features for two scenes that are unrelated to the training dataset.

Furthermore, we utilized a pre-trained ResNet in this task; yet, training the ResNet using scene datasets before integrating it into the scene flow head may have an impact on the outcomes.

Moreover, since there have been notable studies on the fusion of optical flow and scene flow estimation, such as CamLiFlow[8] or a paper about the optical expansion[28], future work for our approach would include integrating optical flows of consecutive scenes besides image features. Thus, optical flow features, in addition to RGB features, may provide useful and distinct features for scene flow estimation which can benefit from them so that we can also investigate its influence on the detection task.

A. Details of Self-Supervised Learning training

A.1. Environment for training

For self-supervised scene flow training, we use one workstation which has a 24GB RTX 3090 GPU. We train our self-supervised scene flow on this GPU. Docker is used to create the training environment. The workstation's operating system is Ubuntu 18.04.6. We generate a new image with CUDA 11.0 and Pytorch 1.7.2 based on the official one since the default CUDA 10.2 in the mmdetetcion3d[37] image does not support the RTX 3090.

A.2. Codes tuning

During the self-supervised scene flow training, we use the scene flow head which is based on FlowNet3D[30]. We cancel the initial skip connection since we use the backbone of PointPillars[3] to replace the feature extraction module in flow network. Moreover, in default mmdetection3d there is method named shuffle which produces random input. However in our self-supervised scene flow training we have two consecutive input therefore we have altered this approach in order to accept random input as pair.

A.2.1. Code tuning for Integration of Image Features from ResNet

Because inputs are delivered into the flow network in the shared backbone, we modified the voxel encoder so that we can use our method for feature extractor. Since we extracted image features from ResNet, we need to combine these with sampled features as we explained in detail in Chapter 3.2.1. The voxel encoder samples random voxelized features and selects 2048 points. ResNet features are 2 dimensional. To integrate ResNet derived image features with sampled features, we add a third dimension and repeat them 2048 times along that dimension.

After concatenation of these features we have more channel numbers than default value which is 64. Thus we updated the flow head configuration of channel numbers,which are 2112 channels, according to new features.

A.2.2. Code tuning for Integration RGB Point Clouds

In mmdetection3d[37], we need to utilize the *LoadImagesFromFile* class to import images from cameras. However, training performance has reduced when using this class, thus we load images from nuScenes on our own and add nuScenes instance before beginning training. Furthermore, in addition to utilizing our implemented projection technique, we changed the voxel encoder in the common backbone. After that, RGB point clouds may be used as input.

A.3. Hyperparameters setting

Our self-supervised scene flow is trained with *samples_per_gpu* set to 2, which implies that two frames are sent into the network in each training step. The batch size equals *samples_per_gpu*, and so in our training, the batch size is 2. The backbone for the PointPillars-based scene flow training is the Adam optimizer with a 0.001 learning rate and voxel encoder. In addition, the number of points to be sampled is N = 2048. There are a total of 12 training epochs according to cycle_loss evaluation.

A.3.1. Setting hyperparameters for the Integrated RGB technique

The hyperparameters must be updated in order to integrate RGB point clouds and images. We update the input modality in PointPillars for nuScenes dataset, which is *use_camera = True*, since images must be loaded. Sweeps in the nuScenes correspond to intermediate frames without annotations in mmdetection3d[37]. Sweeps must be modified for proper projection from point cloud to image, as previously stated, since many sweeps are loaded in default mode. As a result, in the training pipeline, we set *sweeps_num* to 0. Furthermore, since RGB point clouds are used as input in the voxel encoder, the number of channels configurations were changed from 4 to 7, as RGB point clouds contain 7.

B. Details of 3D Object Detection training

B.1. Environment for training

The training environment is identical to that of the Self-supervised Scene Flow training.

B.2. Codes tuning

The default architecture of mmdetection3d[37] is used for the 3D detection task. Weights are initialized from self-supervised task therefore we loaded specified checkpoints in the PointPillars[3] configuration for nuScenes.

B.3. Hyperparameters setting

We also make use of mmdetection3d's default settings. We use the RTX 3090 for the PointPillars[3] training, with *samples_per_gpu* = 4, which implies *batch size* = 4. The initial learning rate is $e - 3$ and decays at the 20th and 23rd epochs with a decay weight of 0.001. As the optimizer, AdamW is utilized. Total training epocs are 24.

List of Figures

List of Tables

Bibliography

[1] E. Erçelik, E. Yurtsever, M. Liu, Z. Yang, H. Zhang, P. Topçam, M. Listl, Y. K. Çaylı, and A. Knoll. "3D Object Detection with a Self-supervised Lidar Scene Flow Backbone". In: *arXiv preprint arXiv:2205.00705* (2022).

[2] X. Chen, L. Kundu, Z. Zhang, H. Ma, S. Fidler, and R. Urtasun. "Monocular 3d object detection for autonomous driving." In: *Proceedings of the IEEE conference on computer vision and pattern recognition* (2016).

[3] A. Lang, V. H., C. S., L. H. Zhou, J. Yang, and O. Beijbom. "Pointpillars: Fast encoders for object detection from point clouds." In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition* (2019), pp. 12697–1270.

[4] K. Bengler, K. Dietmayer, B. Farber, M. Maurer, C. Stiller, and H. Winner. "Three decades of driver assistance systems: Review and future perspectives." In: *IEEE Intelligent Transportation Systems Magazine* (2014).

[5] H. Caesar, V. Bankiti, A. H. Lang, S. Vora, V. E. Liong, Xu, Q., Krishnan, A., Pan, Y., Baldan, G., and Beijbo. "nuscenes: A multimodal dataset for autonomous driving." In: *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (2020), pp. 11621–11631.

[6] A. Geiger, P. Lenz, and R. Urtasun. "Are we ready for autonomous driving? the kitti vision benchmark suite." In: *IEEE conference on computer vision and pattern recognition. IEEE* (2012).

[7] H. Mittal, B. Okorn, and D. Held. "Just go with the flow: Self-supervised scene flow estimation." In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition* (2020), pp. 11177–11185.

[8] H. Liu, T. Lu, Y. Xu, J. Liu, W. Li, and L. Chen. "CamLiFlow: Bidirectional Camera-LiDAR Fusion for Joint Optical Flow and Scene Flow Estimation." In: *arXiv:2111.10502v4* (2021).

[9] Rishav, R. Battrawy, R. Schuster, O. Wasenmüller, and D. Stricker. "DeepLiDARFlow: A Deep Learning Architecture For Scene Flow Estimation Using Monocular Camera and Sparse LiDAR." In: *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)* (2020).

[10] K. He, X. Zhang, S. Ren, and J. Sun. "Deep Resiudal Learning for Image Recognition." In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition.* (2016).

[11] C. R. Qi, H. Su, K. Mo, and L. J. Guibas. "Pointnet: Deep learning on point sets for 3d classification and segmentation." In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition* (2017), pp. 652–660.

[12] J. C. R. Qi, W. Liu, C. Wu, H. Su, and L. J. Guibas. "Frustum pointnets for 3d object detection from rgb-d data." In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition* (2018), pp. 918–927.

[13] Rishav, R. Battrawy, R. Schuster, O. Wasenmüller, and D. Stricker. "Frustum-PointPillars: A Multi-Stage Approach for 3D Object Detection using RGB Camera and LiDAR." In: *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)* (2020).

[14] S. Ren, K. He, R. Girshick, and J. Sun. "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks." In: *Proceedings of the Conference and Workshop on Neural Information Processing Systems* (2015).

[15] Y. Zhou and O. Tuzel. "Voxelnet: End-to-end learning for point cloud based 3d object detection." In: *Proceedings of the IEEE conference on computer vision and pattern recognition* (2018), pp. 4490–4499.

[16] Y. Yan, Y. Mao, and B. Li. "Second: Sparsely embedded convolutional detection." In: (2018).

[17] S. Shi, C. Guo, L. Jiang, Z. Wang, J. Shi, X. Wang, and H. Li. "Pv-rcnn:Point-voxel feature set abstraction for 3d object detection." In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition.* (2020), pp. 10529–10538.

[18] T. Yin, X. Zhou, and P. Krahenbuhl. "Center-based 3d object detection and tracking." In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition.* (2021), pp. 11784–11793.

[19] J. Noh, S. Lee, and B. Ham. "HVPR: Hybrid Voxel-Point Representation for Single-stage 3D Object Detection." In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition.* (2021), pp. 14605–14614.

[20] Z. Zhang, R. Girdhar, A. Joulin, and I. Misra. "Self-Supervised Pretraining of 3D Features on any Point-Cloud." In: *Proceedings of the IEEE/CVF International Conference on Computer Vision* (2021).

[21] S. Huang, Y. Xie, S.-C. Zhu, and Y. Zhu. "Spatio-temporal self-supervised representation learning for 3d point clouds." In: *Proceedings of the IEEE/CVF International Conference on Computer Vision.* (2021), pp. 6535–6545.

[22] J. Hur and S. Roth. "Self-supervised monocular scene flow estimation." In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition* (2020), pp. 7396–7405.

[23] V. Zuanazzi, J. v. Vugt, O. Booij, and P. Mettes. "Adversarial self-supervised scene flow estimation." In: *2020 International Conference on 3D Vision (3DV). IEEE.* (2020), pp. 1049–1058.

[24] S. Baur, D. Emmerichs, F. Moosmann, P. Pinggera, B. Ommer, and A. Geiger. "SLIM: Self-Supervised LiDAR Scene Flow and Motion Segmentation." In: *Proceedings of the IEEE/CVF International Conference on Computer Vision* (2021).

[25] Z. Teed and J. Deng. "RAFT: Recurrent All-Pairs Field Transforms for Optical Flow." In: *Proceedings of European Conference on Computer Vision* (2020).

[26] D. Sun, X. Yang, M.-Y. Liu, and J. Kautz. "Pwc-net: Cnns for optical flow using pyramid, warping, and cost volume." In: *Proceedings of the IEEE conference on computer vision and pattern recognition.* (2018), pp. 8934–8943.

[27] P. Liu, M. Lyu, I. King, and J. Xu. "SelFlow: Self-Supervised Learning of Optical Flow." In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition.* (2019).

[28] G. Yang and D. Ramanan. "Upgrading Optical Flow to 3D Scene Flow through Optical Expansion." In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition* (2020).

[29] L. Shao, P. Shah, V. Dwaracherla, and J. Bohg. "Motion-based Object Segmentation based on Dense RGB-D Scene Flow." In: *IEEE Robotics and Automation Letters 3.4* (2018), pp. 3797–3804.

[30] X. Liu, C. R. Qi, and L. J. Guibas. "Flownet3d: Learning scene flow in 3d point clouds." In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition* (2019), pp. 529–537.

[31] C. R. Qi, L. Yi, H. Su, and L. J. Guibas. "Pointnet++: Deep hierarchical feature learning on point sets in a metric space." In: *Proceedings of the Conference and Workshop on Neural Information Processing Systems* (2017).

[32] W.-C. Ma, S. Wang, R. Hu, Y. Xiong, and R. Urtasun. "Deep rigid instance scene flow." In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition.* (2019), pp. 3614–3622.

[33] Y. Wei, Z. Wang, Y. Rao, J. Lu, and J. Zhou. "PV-RAFT: Point-voxel correlation fields for scene flow estimation of point clouds." In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition* (2021), pp. 6954–6963.

[34] Z. Teed and J. Deng. "Raft-3D: Scene flow using rigid-motion embeddings." In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition* (2021).

[35] W. Wu, Z. Wang, Z. Li, W. Liu, and L. Fuxin. "PointPWC-Net: Cost Volume on Point Clouds for (Self-)Supervised Scene Flow Estimation." In: *Proceedings of European Conference on Computer Vision* (2020).

[36] G. Bradski. "The OpenCV Library". In: *Dr. Dobb's Journal of Software Tools* (2000).

[37] "MMDetection3D: OpenMMLab next-generation platform for general 3D object detection." In: *https://github.com/open-mmlab/mmdetection3d* (2020).

[38] N. Mayer, E. Ilg, P. Hausser, P. Fischer, D. Cremers, A. Dosovitskiy, and T. Brox. "A large dataset to train convolutional networks for disparity, optical flow, and scene flow estimation." In: *Proceedings of the IEEE conference on computer vision and pattern recognition* (2016).